Implementation plan — the open items from live play, 2026-08-01

Findings and evidence: `docs/FINDINGS-2026-08-01-live-play.md`.

Six games were played against MAG in the real Showdown client on the night of 2026-08-01. MAG went **1–2** in the games where the policy was actually driving (games 1–3 had the levers off and test nothing). Everything below is work that live play made visible and that is **not yet done**.

Ordering is by dependency, not by importance: **A blocks C and D.**

A. THE REFIT — blocks C and D

Why. `board.js:2786` changed what `allyHit` *means* without changing its name: it could not see type immunity, so a Flying partner beside Earthquake read as hit. Every shipped weight was fitted against the old meaning. `engine/board.js` is required by 40 files; the fitted vector is read by 24.

Do.

1. Refit the 56-feature vector (`engine/fit_policy.js`) and the 74-weight joint vector (`engine/fit_joint.js`). Both, not one — the joint file embeds the marginal block.
2. Compare `allyHit` before and after. It is currently **−0.0187**, against `deadSide` −2.879 and `abilityBlock` −2.184 in the same vector.
3. Check whether `spreadFreeBesideAlly` moves off zero now that it can fire at all.

The hypothesis this tests. That the immunity bug is *why* `allyHit` came out near zero — the feature averaged free hits (Flying partner, immune) together with costly ones (a Water partner) under one name. Untested until this runs. If `allyHit` stays near zero afterwards, the honest conclusion is that humans genuinely do not avoid hitting their own partner much, and the feature should be reported that way rather than re-explained.

Verify. Paired seed-matched H2H, refit vs shipped, per `engine/sprt.js`. Size it to the question — roughly 200k games confirms a real effect; more is gold-plating.

B. `deadVolatile` — and it CANNOT be fitted the way the others were

Why. The "this move cannot work right now" family is symmetric except for one hole:

feature	catches	fitted from corpus?
`deadStatus`	target already has a status	yes
`deadSide`	side condition already up	yes (−2.879)
`deadField`	field effect already up	yes
`deadWeather`	weather already up	yes
`deadStall`	stalled last turn	yes
`deadNoLastMove`	target has not moved	yes
absent	**target already has this volatile**	**NO — see below**

Observed live: MAG's Whimsicott used **Encore three times, one of which failed outright**. The same defect is already recorded in `board.js:624` in its Taunt form — *"So Taunting into a Taunt looked identical to the first one."*

The constraint that changes the approach. `board.js:620` states plainly that **the stored corpus cannot see volatiles**, and calls it "a real limitation on the FIT". `startVolatile` / `hasVolatile` were added so that *play* could see them, and they work — but no feature consumes them for the dead-move case. So this cannot be handled like `deadSide`: there is no signal in the human replays from which a weight could be learned.

Do — and this is a deliberate departure from how the other six were built. Implement it as a **hard candidate constraint at play time, not a fitted feature.** Re-applying a volatile that is already present does not *tend* to be worse; it **definitionally fails**, which is a rule, not a preference. A fitted weight would be an unidentifiable parameter estimated from data that cannot contain the answer.

if (m.volatileStatus && board.hasVolatile(foeSide, letter, m.volatileStatus)) -> candidate is dead

Uses `hasVolatile(side, letter, name)`, `board.js:639`, which already stores an expiry turn rather than a countdown.

No refit required, because no weight is added.

Verify. Count `|-fail|` events attributable to MAG re-applying a live volatile, before and after, over a fixed self-play seed set. It should go to zero. That is a direction assertion, not a count assertion.

C. `bothSameSideCondition` — pair feature *(needs A first)*

Why. Two Light Screens on the same turn: *"two light screens on the same turn? bro"*. `deadSide` is the most negative weight in the vector and **cannot** help, because it asks whether the condition is already up, and on the turn both slots choose it, it is up for neither. None of the 18 pair features covers it — the closest, `bothStatus`, is about status moves generally.

Do. Add to `JOINT_FEATURES`: fires when both candidate moves carry a `sideCondition` and it is the same one. A dex-field comparison; no move is named.

Refit required — this changes the joint feature *list*, which `magnemite.js:297` correctly refuses to run against a mismatched weight file. That guard will catch it, loudly, which is the behaviour we want.

Open question, do not assume: whether the stored corpus records **both slots' choices on the same turn** in a form the joint fit can use. `engine/fit_joint.js` already fits 18 pair features, so the machinery exists — confirm this specific pairing is observable before adding the feature, or the weight will be estimated from nothing.

D. A feature for being Choice-locked *(needs A first)*

Why. *"it did swap out after i encored it, but not aftre i tricked it."* Of the 56 features, none represents holding an unwanted Choice item. `stallIntoEncore` is about **inflicting** Encore.

Hypothesis to test first, before building anything: that the Encore switch happened because Showdown collapses the legal move list to one — changing the choice space, not the score. A probe that logs the scored candidates on a choice-locked turn settles it. **If that is the cause, this feature may be unnecessary**, because the same collapse happens on a Choice lock too, and the real question becomes why the bot did not act on it.

Do (only if the probe says a feature is needed). Fire when the Pokémon's current item is a Choice item that differs from its sheet item, or more generally when its legal move count has collapsed below its known move count. The second form is broader and covers Encore, Taunt and Disable with one term.

Refit required.

Corpus question, unresolved: whether mid-game item changes (`|-item|` from Trick/Knock Off) are recorded in the stored corpus. If they are not, this has the same problem as B and should likewise be a play-time rule rather than a fitted weight. **Check before building.**

D2. THE BRING NEVER LOOKS AT THE OPPONENT — and may invalidate a published null result

> *Will: "like how does it choose its pokemon before team sheet selection"*

How it currently works. `prior_player.js:154 chooseTeamPreview(team)` draws four species without replacement weighted by `p_bring`, then two of those four weighted by `p_lead`, both measured from ladder behaviour in `engine/bring_priors.js`, sampled off the battle PRNG so the decision is replayable, with a temperature knob for exploration. It degrades to `'default'` if priors are missing and counts that. As a *marginal* sampler it is well built.

Two defects, stacked.

1. **Structural: it has no opponent input.** The signature is `chooseTeamPreview(team)` — its own team and nothing else. The weights are per-species marginals: *how often does anyone bring this Pokémon*, never *should I bring it against THIS team*. MAG plays the same bring distribution against every opponent it will ever face.
2. **A race: the sheet arrives after the choice.** Measured from the live protocol log, battle 26:

acceptopenteamsheets /choose team 4251 <- MAG commits its four showteam|p1 <- the opponent's sheet arrives showteam|p2

So even with open team sheets agreed, the information lands *after* the decision it should inform.

Do.

- Fix the race first, because it is cheap and it is a precondition for anything else: when open team sheets have been accepted, hold the team-preview answer until `|showteam|` for the foe has been seen (with a timeout, falling back to the current sampler and **counting** the fallback).
- Then give the bring an opponent term. This is a genuine modelling project, not a patch, and it should be scoped separately — team selection is a large share of VGC skill.

Check this BEFORE building the opponent term — it may already be answered. This project reports a null result: *better beliefs about the opponent did not improve the bring decision*. If the bring decision never consumed the opponent at all, that null is very close to guaranteed — an input a function does not read cannot improve its output.

Stated as a hypothesis, not a conclusion. It has not been checked how that experiment was run, and it is possible it measured something else (for example a downstream effect of beliefs on play rather than on the bring itself). Read the experiment first. If it did route through this function, the null result says nothing about beliefs and should be withdrawn rather than cited.

E. A support floor for sampled sets — bot only, no refit

Why. *"why smooth rock bro what sort of set is this."* The set was real: 1 of 162 Gyarados sheets (0.62%). Across the corpus, **3,949 of 10,504 distinct sets were seen exactly once — 37.6%.** A set seen once is indistinguishable from a typo, a troll, or a misclick.

Do. In `showdown_bot.js` only, sample within `species_sets.cover(sp, 0.80)` instead of the full tail. Measured cost: Gyarados 38 sets → 12, Incineroar 329 → 48, Grimmsnarl 179 → 27, Glimmora 57 → 5, Politoed 99 → 37.

Explicitly NOT global. DITTO's gauntlet must keep the entire tail — `species_sets.js:23` says it directly: optimising a team against opponents who all run the modal set is optimising against a metagame that does not exist. The two callers want opposite things, and this is a caller's decision.

0.80 is a parameter and is stated as one. It is not derived from anything; it is a starting value. If it matters, sweep it.

F. A guard against a feature's MEANING changing — the structural one

Why. This is the gap that let A happen silently. `magnemite.js:297` compares joined feature names; `:299` compares vector length. **Both pass when a feature quietly starts meaning something else.** Nothing in the project fails when semantics drift under a stable name — and 24 files' worth of weights are invalidated when it does.

Do. Store a hash of every feature's values over a small fixed fixture board, checked when weights load. A changed meaning then fails at load with the feature's name, instead of silently producing numbers fitted against a different definition.

This is the highest-leverage item on the list and the only one that prevents a repeat rather than fixing an instance. It is also the cheapest.

G. Not an implementation item, but the reason several of these exist

Three separate defects on 2026-08-01 had one shape: **knowledge that already existed in the repository, not applied at a new call site.**

defect	what the repo already said	what the new caller did
mega	comment: *"Defaulted to 1"*	default was `0`
capability testing	`test-wiring.js` forces mega and open sheets ON	production passed neither
switching	`mew.js:135` : *"measured as a 10-point LOSS … off until worth more"*	asked for it unconditionally

> ***RETRACTED 2026-08-06 — the 10-point switching loss above is UNATTRIBUTABLE and CONFOUNDED.*** *It was measured through medicham2 playouts on an engine that predates WIRES 123-128, and its artifact carries no* `engine_release` *stamp, so it cannot say which build it ran on. It is also confounded by design:* `bringIn()` *selects* `live(bench)[0]` *, the first healthy body, which the engine itself distinguishes from a switch — "a search that cannot say WHO it is bringing in is not evaluating a switch, it is evaluating LEAVE". Will, 2026-08-06, names the case it cannot represent: out-sped, a lethal hit incoming, and a resist sitting on the bench. See #63 and #57.*

The mega case is the third occurrence of the same defect (`mew.js:446` records the first, across 199,524 games). The pattern is not carelessness at any one site; it is that **defaults are set to the broken value and the working value is supplied by every caller individually**, so a new caller starts from broken. Where a working default exists, it should be the default.

What is deliberately NOT on this list

- **Anything about MAG's playing strength.** Six games were played; three tested wiring only, and the remaining three are 1–2. That is not evidence in either direction and no work should be justified by it.
- **The double Moonblast from game 6.** It looked like overkill and was checked: the first took Sableye to 50/100 and the second finished it. **Correct play.** Recorded here because it was nearly written up as a defect.
- **The immune move in game 5.** That was Will's own Prankster move failing into MAG's Dark-type Sableye, not a MAG error.